IEEE Recommended Practice for Software Requirements Specifications

(IEEE Std 830-1998 structure)

Software Requirements Specification

for

PatchGauge

A Research Toolkit for Measuring Quality, Security, and Maintainability of AI-Assisted Code Changes

Document title	Software Requirements Specification — PatchGauge
Version	1.1 (UML analysis models added)
Prepared by	Aryaman Choudhary
Programme / year	B.Tech. Computer Science and Engineering, Third Year
Organization	Department of Computer Science and Engineering
Document type	Undergraduate R&D project SRS
Date created	06 September 2026
Status	Version 1.1 — analysis models illustrated

Replace the department line with your exact college name and roll number before printing.

Table of Contents

Revision History....3
1. Introduction....3
1.1 Purpose....3
1.2 Document Conventions....3
1.3 Intended Audience and Reading Suggestions....3
1.4 Product Scope....4
1.5 References....4
2. Overall Description....4
2.1 Product Perspective....4
2.2 Product Functions....5
2.3 User Classes and Characteristics....5
2.4 Operating Environment....5
2.5 Design and Implementation Constraints....6
2.6 User Documentation....6
2.7 Assumptions and Dependencies....6
3. External Interface Requirements....6
3.1 User Interfaces....6
3.2 Hardware Interfaces....7
3.3 Software Interfaces....7
3.4 Communications Interfaces....7
4. System Features....7
4.1 Experiment Workspace Management....7
4.2 Repository and Patch Ingestion....8
4.3 Patch Provenance Labelling....8
4.4 Static Quality Analysis....8
4.5 Security-Smell Analysis....9
4.6 Test Presence and Optional Execution....9
4.7 Comparative Reporting and Export....9
4.8 Researcher Dashboard....10
4.9 Reliability of the Analysis Pipeline....10
5. Other Nonfunctional Requirements....10
5.1 Performance Requirements....10
5.2 Safety Requirements....11
5.3 Security Requirements....11
5.4 Software Quality Attributes....11
5.5 Business Rules....11
6. Other Requirements....14
Appendix A: Glossary....15
Appendix B: Analysis Models....15
B.1 Context diagram....15
B.2 Logical data model....16
B.3 Pipeline state model....17
B.4 Top-level use cases....18
B.5 Sequence diagram....19

Appendix C: To Be Determined List.....20

Revision History

Name	Date	Reason for changes	Version
Aryaman Choudhary	06 Sep 2026	Initial SRS for undergraduate software-engineering R&D project	1.0
Aryaman Choudhary	06 Sep 2026	Added UML figures B.1–B.5 to Appendix B (context, class, state, use case, sequence).	1.1

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for PatchGauge version 1.0, a research-oriented software toolkit that helps a student researcher collect, analyse, and compare software patches produced with and without AI coding assistants.

The document covers the complete student-scale system: repository ingestion, patch labelling, static quality and security analysis, optional test execution, comparative reporting, and experiment export. It does not specify a commercial SaaS platform, a new foundation model, or a full replacement for GitHub Copilot, Cursor, or enterprise static-analysis suites.

This SRS is the controlling requirements baseline for design, implementation, testing, and project evaluation of the undergraduate R&D work.

1.2 Document Conventions

- The document follows the section organisation of IEEE Std 830-1998.
- Body text uses Times New Roman. Requirement identifiers are bold.
- Each functional requirement has a unique identifier of the form PG-<AREA>-<NN>, where AREA is IN (ingestion), LB (labelling), QA (quality analysis), SEC (security), TST (testing), RPT (reporting), UX (user interface), or INT (interfaces).
- Priority values are High, Medium, or Low. A High priority is required for a passing project demonstration. Medium items are expected in the final build if time allows. Low items are optional extensions.
- The words shall, should, and may are used in the IEEE sense: shall is mandatory, should is recommended, may is optional.
- Priorities are not inherited automatically. Every requirement states its own priority in Section 4.
- TBD items, if any, are collected in Appendix C.

1.3 Intended Audience and Reading Suggestions

This SRS is written for the following readers:

- The student developer (primary implementer) — read the entire document; use Sections 3 and 4 as the implementation checklist.
- Project guide / faculty evaluator — start with 1.4, 2.1, 2.2, Section 4, and Section 5 to judge scope and evaluability.
- Internal testers or classmates who help with user studies — read 2.3, 3.1, 4.6, and 5.4.
- Future students who may extend the project — read 2.7, Section 6, and the appendices.

Suggested reading order: product scope (1.4) → overall description (Section 2) → system features (Section 4) → external interfaces (Section 3) → non-functional requirements (Section 5).

1.4 Product Scope

PatchGauge is a desktop/CLI research toolkit whose purpose is to answer a measurable software-engineering question:

Do AI-assisted code changes differ from human-written changes in static quality, security-smell density, test presence, and maintainability indicators on a curated set of public repositories?

Objectives of version 1.0:

- Ingest pull requests or commits from selected public GitHub repositories.
- Record whether a change is labelled human-written, AI-assisted, or unknown, using explicit metadata and optional heuristics.
- Run a fixed analysis pipeline (complexity, duplication signals, basic security rules, test-file heuristics).
- Produce a comparative report and a machine-readable dataset that can be attached to the project report.
- Keep the experiment repeatable: same repos, same analysers, same metric definitions.

Out of scope for v1.0: training or fine-tuning large language models; live IDE plug-in inside VS Code; closed-source enterprise codebases; legally binding security certification; fully autonomous program repair.

Academic goal: demonstrate that a 3rd-year CSE student can frame an empirical software-engineering question, build a supporting tool, and evaluate it with defined metrics rather than only a demo application.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2018, Systems and software engineering — Life cycle processes — Requirements engineering (consulted for requirement quality attributes).
- ISO/IEC 25010:2011, Systems and software Quality Requirements and Evaluation (SQuaRE) — used as the quality-attribute vocabulary in Section 5.4.
- GitHub REST API documentation, current public API for repository and pull-request access.
- OWASP Top 10 and common static-analysis rule packs (Bandit for Python, Semgrep community rules) as analysis oracles, not as products being specified.
- Jimenez et al. and subsequent work on Defects4J / software-defect datasets — methodological reference only.
- SWE-bench and related repository-level coding benchmarks — used only as inspiration for repository-level evaluation, not as a required runtime dependency.
- Course project brief and this institution’s undergraduate project guidelines (to be attached by the author).

2. Overall Description

2.1 Product Perspective

PatchGauge is a new, self-contained undergraduate research product. It is not a module of an existing commercial suite. It sits beside ordinary developer tools rather than replacing them.

Context: the student researcher selects public repositories, exports or clones patch history, labels patches, runs analysers, and writes a report. GitHub is an external data source. Optional local tools (Bandit, Semgrep, radon/lizard, pytest/coverage, or language equivalents) are analysis backends. A local SQLite database stores experiment state.

Major components and external interfaces:

- Collector — talks to Git and the GitHub API.
- Label store — records provenance (human / AI-assisted / unknown) and rater notes.
- Analyser farm — wraps static-analysis and test heuristics behind a common result schema.
- Metric engine — aggregates per-file and per-patch scores.
- Reporter — writes Markdown/HTML/CSV/JSON summaries.
- Researcher CLI and a small local web dashboard.

The product is an instrument for empirical software engineering, comparable in spirit to a measurement bench, not to a code generator.

2.2 Product Functions

At a high level the product shall allow the researcher to:

- Configure an experiment (name, language filter, repository list, date window, sample size).
- Clone or fetch selected repositories and extract commits or pull requests as patches.
- Assign or import labels for patch provenance.
- Execute a declared analysis pipeline on each patch.
- Inspect per-patch results and aggregate group comparisons (AI-assisted vs human).
- Export a replication package: raw records, metric table, configuration file, and summary charts.
- Log analyser failures without aborting the whole experiment.

Section 4 specifies these functions as numbered requirements.

2.3 User Classes and Characteristics

User class	Characteristics and importance
Student researcher (primary)	Third-year CSE student who designs the experiment, runs the pipeline, and writes the report. Comfortable with Git, Python, and command-line tools. Must be able to complete a 50–200 patch study on a laptop.
Project guide	Reads exported reports and the replication package. Does not need to run every command. Needs clear metric definitions and limitations.
Lab classmate / rater	May help label a sample of patches as human or AI-assisted. Needs a simple labelling view and short instructions. Occasional user.
Future extender	Another student who adds a language or analyser. Needs documented result schema and plug-in folder. Secondary audience.

The student researcher is the user class that v1.0 must satisfy first. No anonymous public internet users are assumed.

2.4 Operating Environment

- Hardware: a personal laptop or lab PC with at least 8 GB RAM, 20 GB free disk for cloned repositories, and a 64-bit x86_64 or Apple-silicon class CPU. No GPU is required for v1.0.
- Operating systems: Ubuntu 22.04+ or later, Windows 11 with WSL2, or macOS 13+. Primary development target is Ubuntu/WSL.
- Runtime: Python 3.11 or later for the pipeline; Node.js is optional only if the dashboard is implemented as a local static+API UI.
- External services: public GitHub REST API. The system shall operate in offline-replay mode on previously downloaded data when the network is unavailable.

- Coexistence: the toolkit shall not require modification of the user's global Git configuration beyond a documented optional credential helper.

2.5 Design and Implementation Constraints

- Implementation language for the core pipeline shall be Python 3 so that static-analysis wrappers and notebooks stay in one ecosystem.
- Persistent store shall be SQLite for zero-ops deployment. A server database is not required.
- No paid cloud account shall be mandatory. Optional LLM labelling, if used, shall be isolated behind a provider interface and disabled by default.
- Secret handling: GitHub tokens and API keys shall be read from environment variables or a local untracked .env file. They shall never be written into exported reports.
- Only public repositories shall be processed in the default study to avoid institutional ethics and IP issues.
- Analysers may fail on individual files; the pipeline shall isolate those failures.
- The project report must be reproducible from an exported configuration file without undocumented manual clicks.
- Coding standard: PEP 8 for Python, with type hints on public functions.
- Licensing of third-party analysers must remain compatible with academic use (permissive open-source tools preferred).

2.6 User Documentation

The following documentation shall be delivered with the software:

- README.md — install, one-command demo on a bundled sample repo, and common errors.
- EXPERIMENT.md — how to define a study, label patches, and interpret metrics.
- METRICS.md — exact definition of every metric reported (formula, tool, file filters).
- In-CLI help via --help on every command.
- A short 2–4 page user note suitable for attaching to the academic report.

Formats: Markdown in the repository; PDF copies may be generated for submission. No printed manual is required.

2.7 Assumptions and Dependencies

Assumptions:

- The researcher has a GitHub account and can create a personal access token with public-repo read scope.
- Selected repositories build or at least parse sufficiently for static analysis. Full test execution may be limited to a subset of Python projects.
- "AI-assisted" labels will be imperfect. The study will treat labelling error as a validity threat and report it.
- Free-tier GitHub API rate limits are sufficient if the collector caches responses.

Dependencies:

- Git command-line client.
- GitHub REST API availability for live collection.
- Optional analysers: Bandit, Semgrep, lizard or radon, and pytest where tests are run.

- Python packages: requests, gitpython or subprocess Git wrappers, pandas, matplotlib or an equivalent chart library, FastAPI or Flask if a dashboard is included.

If GitHub is unreachable, previously cached JSON and local clones shall still support analysis and reporting.

3. External Interface Requirements

3.1 User Interfaces

PatchGauge v1.0 provides two coordinated interfaces: a command-line interface (required) and a local dashboard (highly recommended).

CLI characteristics:

- A single entry command, patchgauge, with subcommands: init, collect, label, analyze, report, export.
- Progress is printed as per-patch status lines. Failures show repository, patch id, analyser name, and a one-line error.
- Commands that change data ask for confirmation when --yes is absent and the action would overwrite an experiment.
- Help text and error messages shall be in English, complete sentences, and shall not dump raw Python tracebacks to the user by default (tracebacks go to a log file).

Dashboard characteristics (if implemented):

- A local web page served on localhost only (default 127.0.0.1:8765).
- Screens: Experiment overview, Patch table with filters, Patch detail, Comparison charts, Export.
- Every data table shall support sort and a text filter on repository name and patch id.
- Colour is used to encode group (human / AI-assisted / unknown) but all charts shall remain readable in greyscale print.
- No public internet binding shall be the default.

Detailed mock pixel layouts are left to the design document. This SRS constrains behaviour and information content, not visual skin.

3.2 Hardware Interfaces

The product has no special hardware interface. It uses the host file system, standard keyboard/display, and ordinary TCP/IP networking. No sensor, GPU, or microcontroller interface is required.

3.3 Software Interfaces

- Git CLI — clone, fetch, show, diff. Data in: repository URL and commit SHAs. Data out: working trees and unified diffs.
- GitHub REST API — list pull requests, commits, files, and issue text. Authentication via bearer token. The collector shall honour rate-limit headers and back off.
- SQLite 3 — experiment database file stored in the experiment directory.
- Bandit (Python) — JSON report of security issues on changed .py files.
- Semgrep (optional) — JSON report using a documented community ruleset.
- lizard or radon — cyclomatic complexity and function length.
- pytest + coverage.py (optional, Python subset) — test presence and coverage on a shallow checkout of the post-patch revision.
- Local HTTP API between dashboard and pipeline, JSON over HTTP on localhost.

Shared result schema (logical): each analyser writes records {patch_id, analyser, severity, rule_id, file, line, message, extra_json}. The metric engine shall not parse proprietary log text if JSON is available.

3.4 Communications Interfaces

- Outbound HTTPS to api.github.com and github.com for collection.
- Optional outbound HTTPS to an LLM provider only when the researcher explicitly enables AI-assist labelling; default is off.
- Local HTTP for the dashboard.
- No inbound connections from the public Internet are required or permitted by default.
- TLS is provided by the operating system and the requests stack; the product shall not implement custom cryptography.

Message formats are JSON for APIs and UTF-8 text for logs. No real-time streaming protocol is required.

4. System Features

Functional requirements are grouped by feature. Each feature lists description, priority, stimulus/response, and numbered requirements.

4.1 Experiment Workspace Management

4.1.1 Description and Priority

Create and load an experiment folder that contains configuration, the SQLite database, caches, logs, and exports. Priority: High.

4.1.2 Stimulus/Response Sequences

The researcher runs patchgauge init my-study. The system creates the folder, writes a YAML/JSON config skeleton, and initialises an empty database. Later commands accept --experiment PATH.

4.1.3 Functional Requirements

PG-IN-01 The system shall create an experiment workspace with subfolders config/, cache/, data/, logs/, and export/.

PG-IN-02 The system shall store experiment metadata: title, creator, creation time, language filter, and notes.

PG-IN-03 The system shall refuse to overwrite an existing workspace unless --force is provided.

PG-IN-04 The system shall load and validate the configuration file before any collect or analyze run and shall list missing required fields.

PG-IN-05 The system shall record the tool version and analyser versions in the workspace so that a report can state the exact toolchain.

4.2 Repository and Patch Ingestion

4.2.1 Description and Priority

Obtain patches (pull requests or commits) from public repositories and store diffs plus metadata. Priority: High.

4.2.2 Stimulus/Response Sequences

The researcher supplies a list of owner/repo names and a date window. The collector fetches candidate pull requests, downloads diffs, and inserts patch records. On API throttling it waits and continues.

4.2.3 Functional Requirements

PG-IN-06 The system shall accept a list of public GitHub repositories in owner/name form.

PG-IN-07 The system shall extract patches as either pull requests or single commits, as selected in configuration.

PG-IN-08 Each stored patch shall include: repository, external id, title, author login, timestamps, files changed, additions, deletions, and unified diff path.

PG-IN-09 The system shall allow filters on programming language of changed files, minimum/maximum files touched, and date range.

PG-IN-10 The system shall cache API responses under cache/ and reuse them on reruns unless --refresh is set.

PG-IN-11 The system shall stop cleanly when the configured sample size is reached.

PG-IN-12 The system shall skip private or inaccessible repositories and record the skip reason.

PG-IN-13 The collector shall persist a collection log that a later audit can read without rerunning the network.

4.3 Patch Provenance Labelling

4.3.1 Description and Priority

Assign each patch a provenance class used as the independent variable of the study. Priority: High.

4.3.2 Stimulus/Response Sequences

The researcher opens the label command or dashboard table, reviews the PR title/body/diff sample, and sets a label. Alternatively a CSV of labels is imported. The system stores rater id and timestamp.

4.3.3 Functional Requirements

PG-LB-01 The system shall support labels {human, ai_assisted, mixed, unknown}.

PG-LB-02 The system shall store optional free-text rationale and the rater identifier for each label.

PG-LB-03 The system shall import and export labels as CSV with patch_id, label, rater, notes.

PG-LB-04 The system may offer heuristic hints (for example PR body mentions of Copilot, Cursor, ChatGPT, Claude, or “generated by”) but shall not auto-commit a final label without researcher confirmation in v1.0.

PG-LB-05 The system shall compute and display simple inter-rater agreement when two raters label an overlapping sample.

PG-LB-06 Patches labelled unknown shall be excluded from the primary group comparison but retained in the dataset.

4.4 Static Quality Analysis

4.4.1 Description and Priority

Compute maintainability-oriented static metrics on files touched by each patch. Priority: High.

4.4.2 Stimulus/Response Sequences

After collection, the researcher runs patchgauge analyze --suite quality. For each patch the system checks out or reconstructs changed files and runs complexity and size analysers.

4.4.3 Functional Requirements

PG-QA-01 The system shall compute lines added, lines deleted, files touched, and number of hunks per patch.

PG-QA-02 The system shall compute cyclomatic complexity for supported languages on changed functions when the analyser can parse the file.

PG-QA-03 The system shall record functions whose complexity exceeds a configurable threshold (default 10).

PG-QA-04 The system shall flag generated-looking boilerplate signals: large blocks of repeated identical lines, empty except blocks, and TODO/FIXME introduced by the patch.

PG-QA-05 The system shall skip binary files and vendor/minified directories using a documented path ignore list.

PG-QA-06 When an analyser cannot parse a file, the system shall store a structured error and continue.

PG-QA-07 All quality metrics shall be defined in METRICS.md and referenced by name in the exported table.

4.5 Security-Smell Analysis

4.5.1 Description and Priority

Run lightweight static security rules on changed source files and count findings by severity. Priority: High for Python; Medium for additional languages.

4.5.2 Stimulus/Response Sequences

The analyze command invokes Bandit and/or Semgrep on the after-image of changed files and stores findings.

4.5.3 Functional Requirements

PG-SEC-01 The system shall run Bandit on changed Python files when Bandit is installed.

PG-SEC-02 The system shall store each finding with rule id, severity, file, and line.

PG-SEC-03 The system shall compute findings per thousand added lines as a normalised rate.

PG-SEC-04 The system should run Semgrep with a documented ruleset when Semgrep is installed.

PG-SEC-05 The system shall not claim that a clean static scan proves the patch is secure.

PG-SEC-06 High-severity findings shall be listed individually in the patch detail view.

4.6 Test Presence and Optional Execution

4.6.1 Description and Priority

Estimate whether the patch is accompanied by tests. Optionally execute tests on a Python subset. Priority: Medium (heuristics High, execution Medium).

4.6.2 Stimulus/Response Sequences

The analyser inspects whether the patch adds or modifies files matching test patterns. For configured Python repos it may run pytest with a timeout.

4.6.3 Functional Requirements

PG-TST-01 The system shall detect added or modified test files using configurable glob patterns (default **/test_*.py, **/*_test.py, **/*Test.java, **/*.spec.ts).

PG-TST-02 The system shall record a boolean has_test_change per patch.

PG-TST-03 The system may execute project tests on a whitelist of repositories with a configurable timeout (default 120 seconds) and record pass, fail, or timeout.

PG-TST-04 Test execution shall be disabled by default so that a first demo does not require building every project.

PG-TST-05 The system shall never execute untrusted install scripts unless the researcher opts in with --allow-install.

4.7 Comparative Reporting and Export

4.7.1 Description and Priority

Turn raw measurements into the tables and figures needed by the academic report. Priority: High.

4.7.2 Stimulus/Response Sequences

The researcher runs patchgauge report. The system writes a summary Markdown file, CSV metric table, and at least two charts: metric distributions by group and a bar comparison of mean rates.

4.7.3 Functional Requirements

PG-RPT-01 The system shall compute group statistics (count, mean, median, min, max) for each metric split by provenance label.

PG-RPT-02 The system shall export CSV and JSON files containing one row per patch and one row per finding.

PG-RPT-03 The system shall generate a Markdown summary that states sample sizes and warns when a group has fewer than 20 patches.

PG-RPT-04 The system shall generate at least two charts as PNG files suitable for insertion into the project report.

PG-RPT-05 The system should compute a simple non-parametric comparison (for example Mann–Whitney U) when SciPy is available, and shall label it exploratory.

PG-RPT-06 The export package shall include config, metric definitions hash or version, and a README that explains how to rerun reporting from stored data.

PG-RPT-07 Exports shall omit secrets, tokens, and absolute local home-directory paths where feasible.

4.8 Researcher Dashboard

4.8.1 Description and Priority

Optional localhost UI for browsing patches and charts. Priority: Medium.

4.8.2 Stimulus/Response Sequences

The researcher runs patchgauge ui, opens the printed URL, filters the patch table, and opens a patch detail page showing diff excerpt and findings.

4.8.3 Functional Requirements

PG-UX-01 The dashboard shall display experiment name, patch counts by label, and last analysis time.

PG-UX-02 The dashboard shall list patches with columns: id, repo, title, label, files, findings, has_test_change.

PG-UX-03 The patch detail view shall show metadata, label controls, finding list, and a truncated diff.

PG-UX-04 The dashboard shall not be reachable on 0.0.0.0 unless --expose is passed (default deny).

PG-UX-05 If the dashboard is omitted from the final build, the CLI report shall still satisfy all PG-RPT requirements.

4.9 Reliability of the Analysis Pipeline

4.9.1 Description and Priority

Keep long runs recoverable. Priority: High.

4.9.2 Stimulus/Response Sequences

If the process is interrupted, a subsequent analyze run resumes unfinished patches rather than starting from zero.

4.9.3 Functional Requirements

PG-IN-14 The system shall treat analysis as idempotent per patch+analyser pair and skip completed pairs on rerun.

PG-IN-15 The system shall write structured JSON lines to logs/pipeline.log.

PG-IN-16 The system shall provide patchgauge status that prints counts of collected, labelled, analysed, and failed patches.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

PG-NF-01 On a reference laptop (8 GB RAM), collection of 100 public pull-request metadata records shall complete within 15 minutes under normal GitHub rate limits when cache is cold.

PG-NF-02 Static analysis of 100 already-downloaded Python patches shall complete within 30 minutes without test execution.

PG-NF-03 The dashboard, if present, shall render the patch table of 200 rows within 2 seconds after the local API responds.

PG-NF-04 Peak additional disk use beyond cloned repositories shall stay under 500 MB for metadata, logs, and reports of a 200-patch study.

PG-NF-05 A single patch analyser failure shall not increase end-to-end runtime of the remaining queue by more than the configured per-analyser timeout (default 60 seconds).

5.2 Safety Requirements

PatchGauge is not a safety-critical control system. Residual safety concerns are limited to executing untrusted project code during optional test runs.

PG-NF-06 Optional test execution shall run only when explicitly enabled and should prefer an isolated working directory.

PG-NF-07 The system shall not invoke destructive Git commands (reset --hard on user repositories outside the workspace, push, or force-push).

PG-NF-08 The default workspace shall live under a project directory, not in the user’s home root.

5.3 Security Requirements

PG-NF-09 GitHub tokens and API keys shall not be logged, committed, or copied into export/.

PG-NF-10 The default dashboard bind address shall be 127.0.0.1.

PG-NF-11 The system shall process only repository URLs that pass a simple allow-list pattern (github.com public HTTPS/SSH URLs) in v1.0.

PG-NF-12 Downloaded diffs and findings may contain secrets that were already in public code; the documentation shall warn the researcher not to republish raw diffs if a secret is spotted.

PG-NF-13 No user-account registration subsystem is required; v1.0 is single-operator on a local machine.

5.4 Software Quality Attributes

Quality attributes use ISO/IEC 25010 vocabulary and are written so a guide can test them.

Attribute	Requirement
Functional correctness	Metric values for a bundled 5-patch fixture shall match the checked-in expected CSV within rounding of 0.01.

Reliability	A full pipeline run on the fixture shall exit code 0 even if one synthetic file is unparsable.
Reproducibility	Two report generations from the same database without --refresh shall produce the same metric CSV checksum.
Usability	A classmate following README.md shall complete the fixture demo in under 20 minutes without extra oral help.
Maintainability	A new analyser shall be addable as one Python module plus a config entry, without editing the reporter core.
Portability	Core CLI commands shall run on Ubuntu and WSL2 with the same user-visible options.
Honesty / scientific integrity	Reports shall display sample size and shall not use causal language (“AI causes bugs”) in generated text.

5.5 Business Rules

These operating rules constrain use of the product in an academic setting.

- Only public data is used unless the department later issues a written ethics approval for private repos.
- Generated reports are research instruments, not vendor security audits.
- The student remains responsible for interpreting statistics; the tool shall not auto-generate a “paper abstract” that overclaims.
- Third-party analyser licences must be preserved in a NOTICE or similar file.
- Course plagiarism rules apply to the study write-up; the tool’s export does not replace citation of prior work.

6. Other Requirements

6.1 Data Requirements

PG-INT-01 The SQLite schema shall include tables for experiments, repositories, patches, labels, analysis_runs, findings, and metrics.

PG-INT-02 Patch identifiers shall be stable: repo + source + external id.

PG-INT-03 The bundled fixture dataset shall contain at least five anonymised or public-domain patches covering both labels.

6.2 Internationalisation

v1.0 UI and messages are English only. Source files in any UTF-8 encoding shall be accepted; analyser failures due to encoding shall be logged.

6.3 Legal and Ethical Requirements

The product shall document that mined GitHub data remains under the original project licences. The student shall not redistribute full clones as part of the submission if the course only needs metric tables and a few examples.

Human labelling of patches is low-risk document review. If classmates are timed in an onboarding study extension, participation shall be voluntary.

6.4 Reuse Objectives

The result schema and METRICS.md are intended for reuse by a follow-on student who adds Java analysers or a second research question (for example energy of CI runs). The dashboard is not the reuse boundary; the database is.

6.5 Research Evaluation Requirements

These items are part of the academic product, not extra documentation fluff.

PG-RPT-08 The final project shall analyse at least 80 labelled patches, with a documented attempt to balance groups, or shall explicitly justify a smaller sample.

PG-RPT-09 The report shall include a limitations section covering labelling error, public-repo bias, static-analysis false positives, and language mix.

PG-RPT-10 The demonstration shall include the fixture run (correctness) and one live or recorded run on real public repositories.

Appendix A: Glossary

Term	Definition
AI-assisted patch	A commit or pull request whose author or description indicates substantial use of an AI coding assistant, or which a human rater labels as such under the study protocol.
Analyser	A wrapped external or internal program that emits structured findings or metrics for one patch.
Experiment workspace	The directory that holds configuration, database, caches, logs, and exports for one study.
Finding	A single static-analysis or heuristic alert tied to a file and optional line number.
Human patch	A change labelled as written primarily without AI assistance.
Patch	The unit of analysis: a pull request or commit together with its diff and metadata.
Provenance	The labelled origin class of a patch (human, ai_assisted, mixed, unknown).
Replication package	Exported config, data tables, metric definitions, and instructions sufficient to regenerate the report from stored data.
SQuaRE / ISO 25010	International quality-model vocabulary used in Section 5.4.
Static analysis	Inspection of code without necessarily executing the program.
SRS	Software Requirements Specification.

Appendix B: Analysis Models

This appendix records the analysis models used while writing the requirements. Narrative descriptions are retained so the figures can be regenerated; the diagrams are the normative visual form of each model.

B.1 Context diagram

External entities: Researcher, GitHub, optional analyser binaries, optional test runner. PatchGauge is the single central system. Data stores live inside the experiment workspace (SQLite database, cache, logs, and exports). Flows: repository list in; patches and diffs in; labels in; findings and reports out.

Figure B.1 — System context diagram (UML)

PatchGauge and its external entities

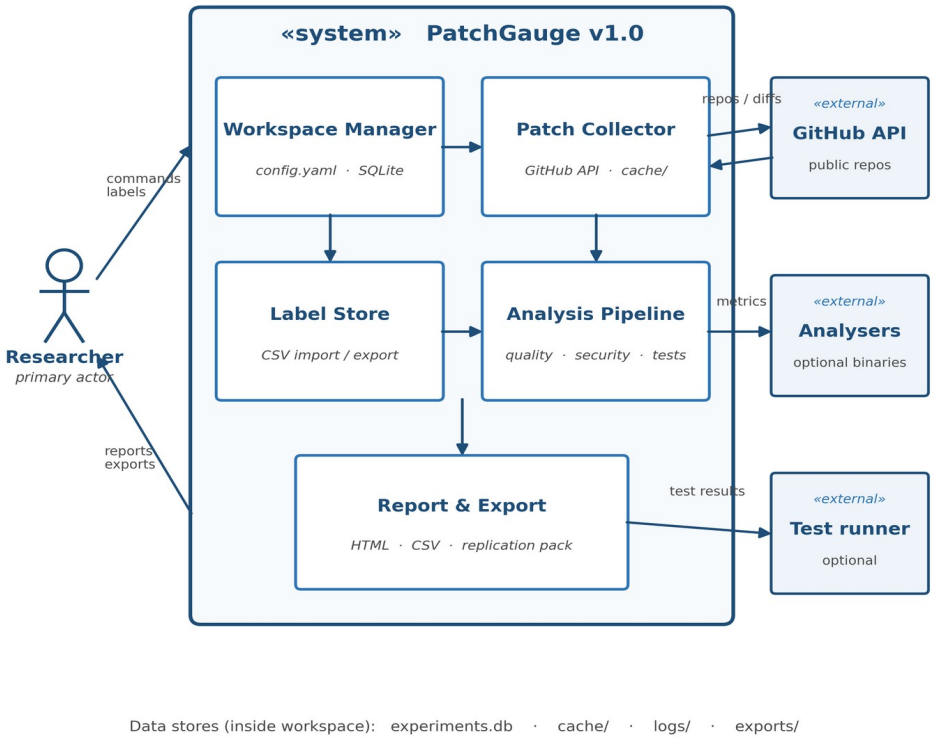


Figure B.1 — System context diagram showing PatchGauge, the researcher, and external services.

B.2 Logical data model

Experiment 1—N Repository; Repository 1—N Patch; Patch 1—N Label; Patch 1—N AnalysisRun; AnalysisRun 1—N Finding; Patch 1—N MetricValue. Primary keys are integers; the business key of a patch is the stable natural identifier defined in PG-INT-02. Provenance is modelled as an enumeration {human, ai_assisted, mixed, unknown}.

Figure B.2 — Logical data model (UML class diagram)

Persistent entities required by PG-INT-01 and PG-INT-02

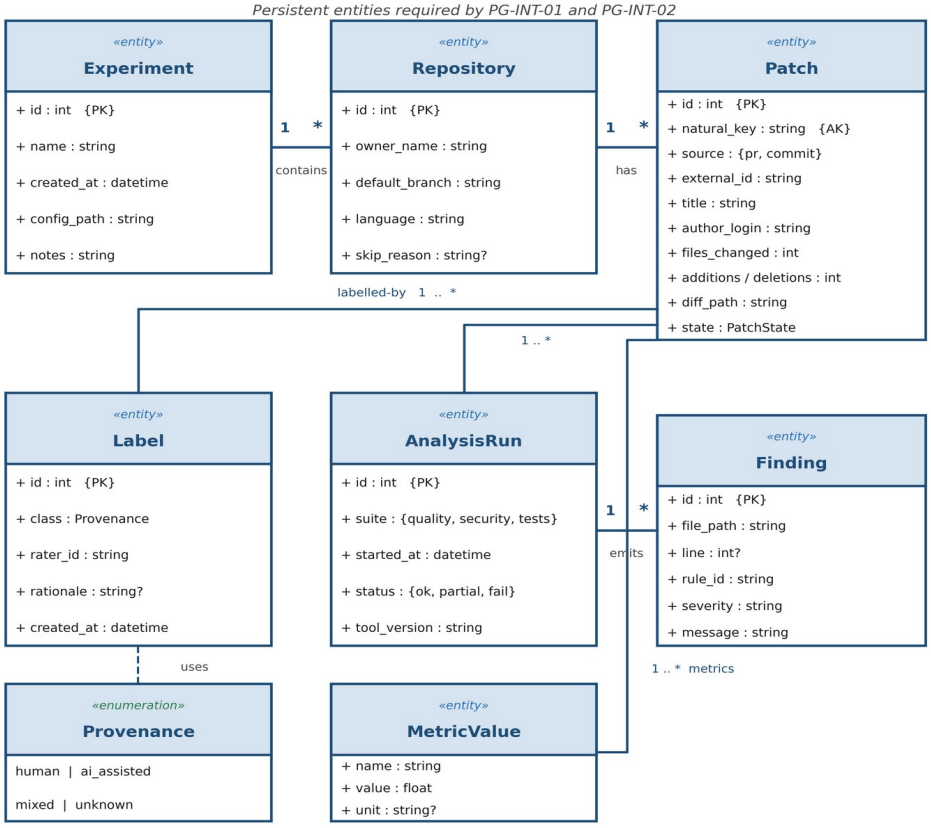


Figure B.2 — Logical data model (UML class diagram) of workspace entities.

B.3 Pipeline state model

A patch moves through states: discovered → collected → labelled → queued → partial / complete → in_report. Terminal side states are skipped and failed_isolated. Report generation reads only labelled patches that have reached analysis complete. Isolated analyser crashes must not abort the remainder of the experiment.

Figure B.3 — Patch lifecycle (UML state machine)

Report generation reads only labelled patches that reached analysis complete

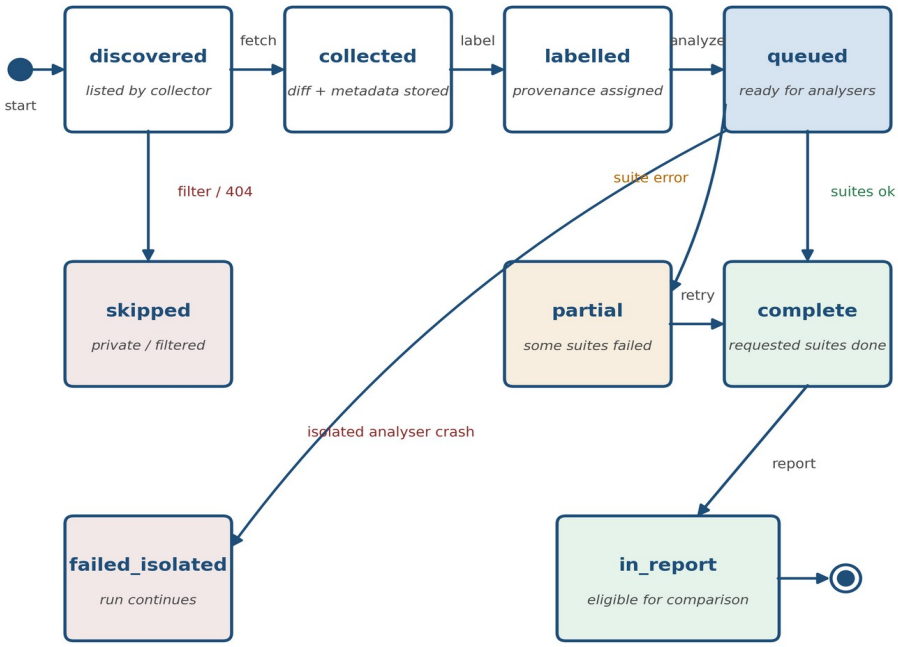


Figure B.3 — UML state machine for the lifecycle of one patch.

B.4 Top-level use cases

- UC1 Initialise experiment
- UC2 Collect patches
- UC3 Label provenance
- UC4 Run analysis suite
- UC5 Inspect patch detail
- UC6 Generate comparison report
- UC7 Export replication package

These use cases map directly to the features in Section 4. GitHub is included by UC2; analysers and the optional test runner are included by UC4.

Figure B.4 — Top-level use cases (UML)

Primary actor: Researcher · supporting actors: GitHub, Analysers, Test runner

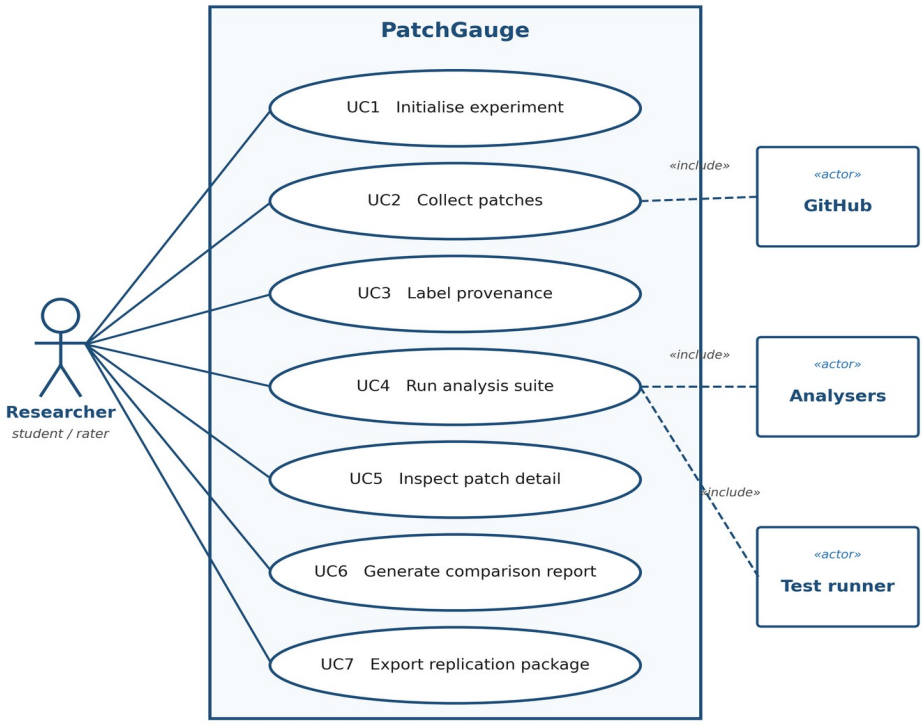


Figure B.4 — UML use-case diagram for PatchGauge v1.0.

B.5 Sequence diagram

The happy-path interaction for a first experiment is: initialise workspace, collect patches from GitHub, import provenance labels, run the quality and security suites, then emit the comparison report and replication package. Return messages are dashed.

Figure B.5 — Sequence: collect, label, analyse, report

Happy-path interaction among the researcher and PatchGauge components

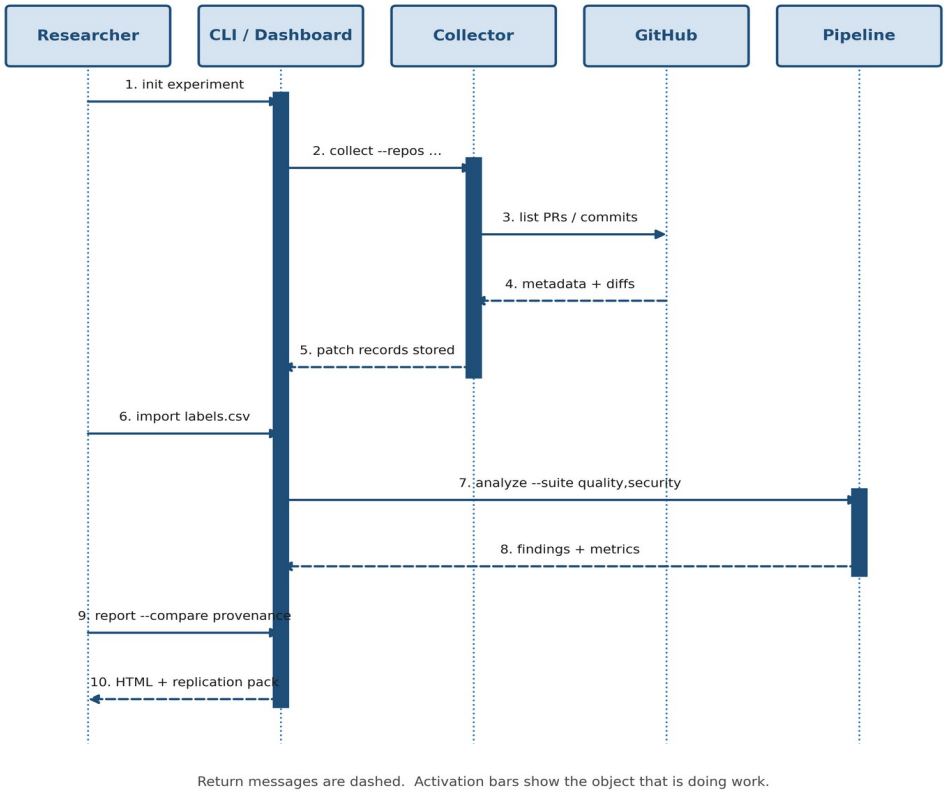


Figure B.5 — UML sequence diagram for collect, label, analyse, and report.

Appendix C: To Be Determined List

ID	Item	Resolution plan
TBD-1	Exact college name, roll number, and guide name on the cover.	Fill before binding the submission copy.
TBD-2	Final language scope if the guide requires Java or JavaScript in addition to Python.	Decide at first review; v1.0 quality suite is Python-first.
TBD-3	Whether the localhost dashboard is mandatory for the viva demonstration.	If the guide says yes, PG-UX-* become High; otherwise CLI+HTML report suffices.
TBD-4	Department ethics wording for mining public GitHub data.	Copy the department's standard public-data paragraph if one exists.

End of Software Requirements Specification — PatchGauge v1.1